



INSTITUTO FED. DE EDUCAÇÃO, CIÊNC. E TEC. DE PERNAMBUCO
CURSO: TEC. EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS
PROFESSOR: RAMIDE DANTAS
ASSUNTO: FIREBASE - FIRESTORE DATABASE

Prática 06

Atenção: Esta prática é continuação da Prática 05. Use controle de versões

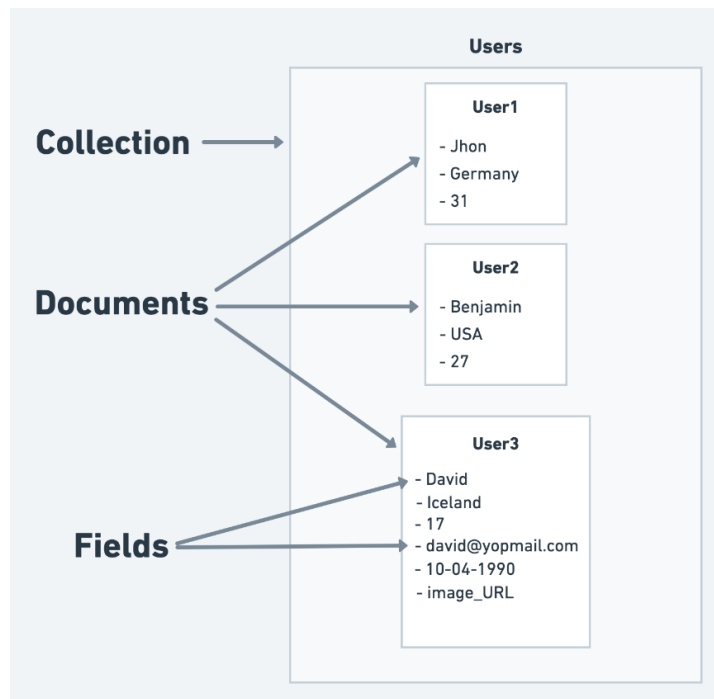
Parte 1: Preparação para usar o Firebase Firestore

Passo 1: No Android Studio, adicione suporte ao Firebase Firestore: *Tools > Firebase*.

Selecione a opção *Cloud Firestore* e depois *Get started* Execute o passo 2 nessa nova aba (o 1º passo já deve estar feito pela Prática 05). Serão adicionadas as dependências necessárias ao *script gradle* da aplicação. Os demais passos são trechos de código para manipulação dos dados.

Passo 2: Habilitando e visualizando a base de dados no Firebase Console:

No [Firebase Console](#), crie um banco de dados do tipo Cloud Firestore no projeto da prática (Criação > *Firestore Database*). Se perguntado(a) sobre permissões, deixe aberto para gravação e leitura (“**modo de teste**”); as regras de acesso devem ser revistas numa aplicação em produção. O *Firestore Database* organiza os dados na forma de coleções de documentos, que podem apontar para outras coleções, e assim sucessivamente.



<https://docs.flutterflow.io/data-and-backend/firebase/firestore-database-cloud-firestore/creating-collections>

Passo 3: Faça o build, rode e teste a aplicação. Dê commit se não houver problemas.

Parte 2: Criando as classes do banco de dados

Passo 1: Crie a classe `User` no pacote `model` com o código:

```
data class User(val name: String, val email: String)
```

Essa classe modela o usuário atualmente logado na aplicação.

Passo 2: Declare um objeto `User` dentro de `MainViewModel`:

```
private val _user = mutableStateOf<User?> (null)
val user : User?
    get() = _user.value
```

Passo 3: Em `MainActivity`, mude o título da `topBar` para incluir o nome do usuário:

```
title = {
    val name = viewModel.user?.name?:"[carregando...]"
    Text("Bem-vindo/a! $name")
}
```

Passo 4: Crie a classe `FBCity` no novo pacote `db.fb`:

```
class FBCity {
    var name : String? = null
    var lat : Double? = null
    var lng : Double? = null

    fun toCity(): City {
        val latlng = if (lat!=null&&lng!=null) LatLng(lat!!, lng!!) else null
        return City(name!!, weather = null, location = latlng)
    }
}

fun City.toFBCity() : FBCity {
    val fbCity = FBCity()
    fbCity.name = this.name
    fbCity.lat = this.location?.latitude ?: 0.0
    fbCity.lng = this.location?.longitude ?: 0.0

    return fbCity
}
```

Essa classe é usada para serializar as cidades no Firebase Firestore. Ela precisa ter construtor *default* vazio e atributos “setáveis” que podem ser nulos. Também adicionamos métodos para transforma de/para `model.City`.

Passo 5: Crie a classe `FBUser` em `db.fb`:

```
class FBUser {
    var name : String ? = null
    var email : String? = null
    fun toUser() = User(name!!, email!!)
}

fun User.toFBUser() : FBUser {
    val fbUser = FBUser()
    fbUser.name = this.name
    fbUser.email = this.email
    return fbUser
}
```

Passo 6: Rode e teste para ver se nada foi quebrado. Faça um novo commit.

Parte 3: Criando o componente de acesso ao BD do Firebase

Passo 1: Crie o objeto `FBDatabase` em `db.fb`:

```
class FBDatabase {
    interface Listener {
        fun onUserLoaded(user: FBUser)
        fun onUserSignOut()
        fun onCityAdded(city: FBCity)
        fun onCityUpdated(city: FBCity)
        fun onCityRemoved(city: FBCity)
    }

    private val auth = Firebase.auth
    private val db = Firebase.firestore
    private var citiesListReg: ListenerRegistration? = null
    private var listener : Listener? = null

    init { ... }

    fun setListener(listener: Listener? = null) { ... }

    fun register(user: FBUser) { ... }

    fun add(city: FBCity) { ... }

    fun remove(city: FBCity) { ... }
}
```

Essa classe nos fornecerá acesso ao nosso BD no Firebase. Ela define um *listener* que é chamado sempre que houver um evento de cidade adicionada ou removida (`onCityAdded/onCityUpdated/onCityRemoved`), ou quando as informações do usuário logado forem carregadas (`onUserLoaded`). Vamos instanciar essa classe nas atividades e setar o `viewModel` como `listener`.

Passo 2: Em `FBDatabase`, use o código baixo no bloco `init`:

```
init {
    auth.addAuthStateListener { auth ->
        if (auth.currentUser == null) {
            citiesListReg?.remove()
            listener?.onUserSignOut()
            return@addAuthStateListener
        }

        val refCurrUser = db.collection("users").document(auth.currentUser!!.uid)
        refCurrUser.get().addOnSuccessListener {
            it.toObject(FBUser::class.java)?.let { user ->
                listener?.onUserLoaded(user)
            }
        }

        citiesListReg = refCurrUser.collection("cities")
            .addSnapshotListener { snapshots, ex ->
                if (ex != null) return@addSnapshotListener

                snapshots?.documentChanges?.forEach { change ->
                    val fbCity = change.document.toObject(FBCity::class.java)
                    if (change.type == DocumentChange.Type.ADDED) {
                        listener?.onCityAdded(fbCity)
                    } else if (change.type == DocumentChange.Type.MODIFIED) {
                        listener?.onCityUpdated(fbCity)
                    } else if (change.type == DocumentChange.Type.REMOVED) {
                        listener?.onCityRemoved(fbCity)
                    }
                }
            }
    }
}
```

Esse código é chamado na criação do `FBDatabase`. Ele registra um (outro) *listener* que é disparado quando o usuário faz login ou logout. Em caso de login, é iniciada a carga dos dados do usuário atual, a qual ao ser completada dispara o evento `listener?.onUserLoaded(...)` (a `?` indica que o *listener* pode ser *null*, nesse caso nada é feito). Também é registrado o *listener* `citiesListReg` que notifica sempre que uma cidade for adicionada (`onCityAdded()`) ou removida (`onCityRemoved()`) no BD. Quando o usuário faz *sign out*, o `citiesListReg` registrado é removido.

Passo 3: Implemente o método `FBDatabase.setListener()` como baixo():

```
fun setListener(listener: Listener? = null) {
    this.listener = listener
}
```

Passo 4: Implemente o método `FBDatabase.register()`:

```
fun register(user: FBUser) {
    if (auth.currentUser == null)
        throw RuntimeException("User not logged in!")
    val uid = auth.currentUser!!.uid
    db.collection("users").document(uid + "").set(user);
}
```

Esse método salva um objeto `FBUser` na coleção `users` no Firebase Firestore e deve ser chamado em `RegisterPage` (ver mais a frente).

Passo 5: Implemente o método `FBDatabase.add()`:

```
fun add(city: FBCity) {
    if (auth.currentUser == null)
        throw RuntimeException("User not logged in!")
    if (city.name == null || city.name!!.isEmpty())
        throw RuntimeException("City with null or empty name!")

    val uid = auth.currentUser!!.uid
    db.collection("users").document(uid).collection("cities")
        .document(city.name!!).set(city)
}
```

Nesse método, caso haja um usuário logado, é criado um registro para a cidade na coleção `cities` do usuário atual, com o nome da cidade como *id*.

Passo 6: Implemente o método `FBDatabase.remove()`:

```
fun remove(city: FBCity) {
    if (auth.currentUser == null)
        throw RuntimeException("User not logged in!")
    if (city.name == null || city.name!!.isEmpty())
        throw RuntimeException("City with null or empty name!")

    val uid = auth.currentUser!!.uid
    db.collection("users").document(uid).collection("cities")
        .document(city.name!!).delete()
}
```

Nesse método, a cidade é removida da lista de cidades associada ao usuário atualmente logado.

Passo 7: Compile para ver se não há problemas.

Não deve fazer diferença ao rodar nesse ponto. Faça um novo *commit* caso esteja tudo certo.

Parte 4: Conectando a UI com o Firebase DB via ViewModel.

Passo 1: Modifique `MainViewModel` de forma a implementar `FBDatabase.Listener`:

```
class MainViewModel (private val db: FBDatabase): ViewModel(),
FBDatabase.Listener {
    private val _cities = mutableStateListOf<City>()
    val cities
        get() = _cities.toList()

    private val _user = mutableStateOf<User?> (null)
    val user : User?
        get() = _user.value

    init {
        db.setListener(this)
    }

    fun remove(city: City) {
        db.remove(city.toFBCity())
    }

    fun add(name: String, location : LatLng? = null) {
        db.add(City(name = name, location = location).toFBCity())
    }

    override fun onUserLoaded(user: FBUser) {
        _user.value = user.toUser()
    }

    override fun onUserSignOut() {
        //TODO("Not yet implemented")
    }

    override fun onCityAdded(city: FBCity) {
        _cities.add(city.toCity())
    }

    override fun onCityUpdated(city: FBCity) {
        //TODO("Not yet implemented")
    }

    override fun onCityRemoved(city: FBCity) {
        _cities.remove(city.toCity())
    }
}
```

`MainViewModel` vai tratar os eventos de adição/remoção de cidades e carga do usuário do `FBDatabase`. Nesse caso, a cidade é adicionada ou removida à lista de cidades (`_cities`), e o usuário é configurado na propriedade `_user`.

Passo 2: Adicione a classe `MainViewModelFactory` abaixo:

```
class MainViewModelFactory(private val db : FBDatabase) :
ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(MainViewModel::class.java)) {
            return MainViewModel(db) as T
        }
        throw IllegalArgumentException("Unknown ViewModel class")
    }
}
```

A classe pode ficar no mesmo arquivo do `MainViewModel`. Essa classe será usada para instanciar o `ViewModel` com o parâmetro correto na `MainActivity`.

Passo 3: Modifique a `MainActivity` como abaixo:

```
class MainActivity : ComponentActivity() {
    @OptIn(ExperimentalMaterial3Api::class)
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            val fbDB = remember { FirebaseDatabase() }
            val viewModel : MainViewModel = viewModel(
                factory = MainViewModelFactory(fbDB)
            )
            ...
        }
    }
}
```

Essa mudança usa o `Factory` para passar o `FirebaseDatabase` para o `viewModel`.

Passo 4: Salvando os dados do usuário em `RegisterPage`:

Adicione o trecho abaixo após o registro bem-sucedido do usuário:

```
fbDatabase().register(User(name, email).toFBUser())
```

Nesse código, usamos uma instância temporária do `FirebaseDatabase` para criar o usuário no banco de dados do Firebase.

Passo 5: Rode e teste a aplicação.

Registre um novo usuário e adicione e remova cidades usando o mapa e o diálogo.

ATENÇÃO: testar com usuários criados na prática 5 deve causar problemas por falta de dados desse usuário no BD.

Faça um *commit* se estiver tudo correto.